

Projet Intégré
Analyse et Prédiction du Churn Client

Guide Étudiant — Démarche et Outils par Étape

Encadrant

2026

Table des matières

1	Guide Étudiant — Démarche et Outils par Étape	2
1.1	Étape 0 — Mise en place de l’environnement	2
1.1.1	Objectifs	2
1.1.2	À faire	2
1.1.3	Outils proposés	2
1.2	Étape 1 — Exploration des données (EDA)	2
1.2.1	Objectifs	2
1.2.2	À faire	3
1.2.3	Outils proposés	3
1.3	Étape 2 — ETL (Extract – Transform – Load)	3
1.3.1	Objectifs	3
1.3.2	À faire	3
1.3.3	Outils proposés	3
1.4	Étape 3 — Data Warehouse et modélisation dimensionnelle (BI)	4
1.4.1	Objectifs	4
1.4.2	À faire	4
1.4.3	Outils proposés	5
1.5	Étape 4 — Tableaux de bord Power BI	5
1.5.1	Objectifs	5
1.5.2	À faire	5
1.5.3	Outils proposés	5
1.6	Étape 5 — Machine Learning	6
1.6.1	Objectifs	6
1.6.2	À faire	6
1.6.3	Outils proposés	6
1.7	Étape 6 — Interface web et déploiement	6
1.7.1	Objectifs	6
1.7.2	À faire	7
1.7.3	Outils proposés — Front + back unifié (recommandé pour ce projet)	7
1.7.4	Outils proposés — Front et back séparés (si vous voulez aller plus loin)	7
1.7.5	Outils proposés — Hébergement / Déploiement	7
1.8	Étape 7 — Rapport et présentation	8
1.8.1	Objectifs	8
1.8.2	Structure recommandée du rapport (25–40 pages)	8
1.8.3	Outils proposés	8
1.8.4	Conseils pour la présentation orale	8
1.9	Récapitulatif — Stack recommandée (option simple et rapide)	8

1. Guide Étudiant — Démarche et Outils par Étape

Ce guide vous accompagne **étape par étape** dans la réalisation du projet. Pour chaque phase, vous trouverez :

- Les **objectifs** à atteindre.
- Les **livrables** intermédiaires attendus.
- Une **proposition d'outils** parmi lesquels choisir (vous restez libres, mais justifiez vos choix dans le rapport).

Conseil général : ne cherchez pas à utiliser les outils les plus complexes. Privilégiez ceux que vous maîtrisez ou que vous pouvez prendre en main rapidement. Un outil simple bien utilisé vaut mieux qu'un outil sophistiqué mal utilisé.

1.1 Étape 0 — Mise en place de l'environnement

1.1.1 Objectifs

- Préparer un environnement de travail reproductible.
- Mettre en place le dépôt GitHub et la collaboration.

1.1.2 À faire

- Créer un dépôt **GitHub** privé pour la phase de développement, public à la livraison (ou public dès le départ si vous excluez bien les données).
- Inviter l'encadrant en tant que collaborateur.
- Créer un environnement virtuel Python (**venv** ou **conda**).
- Rédiger un **requirements.txt** ou **environment.yml**.
- Rédiger un premier **README.md** avec : description, structure du dépôt, instructions d'installation.
- Configurer un **.gitignore** qui exclut **toutes les données** (*.csv, *.xlsx, *.pkl, dossier **data/**, etc.) et les fichiers d'environnement (**venv/**, **__pycache__/**, **.env**).

1.1.3 Outils proposés

Catégorie	Options
Versioning	Git + GitHub (obligatoire)
IDE	VS Code, PyCharm, JetBrains DataSpell
Environnement Python	venv , conda , mamba , uv , poetry
Notebooks	Jupyter Lab, Jupyter Notebook, VS Code Notebooks
Gestion de projet	GitHub Projects, Trello, Notion

1.2 Étape 1 — Exploration des données (EDA)

1.2.1 Objectifs

- Comprendre la structure, la volumétrie et la qualité des données.

- Identifier les variables intéressantes pour le churn.
- Documenter les problèmes de qualité à traiter.

1.2.2 À faire

- Charger les fichiers et vérifier leur intégrité.
- Calculer : nombre de lignes, types des colonnes, taux de valeurs manquantes, distributions.
- Croiser les données avec les dimensions (vérifier que les clés correspondent).
- Visualiser : distributions par variable, corrélations, taux de churn par segment.
- Produire un **rapport d'exploration** (notebook) avec vos premières observations.

1.2.3 Outils proposés

Catégorie	Options
Manipulation	pandas, polars (plus rapide sur grosse volumétrie), DuckDB
Profilage automatique	ydata-profiling (ex-pandas-profiling), sweetviz , dataprep
Visualisation	matplotlib, seaborn, plotly, altair

1.3 Étape 2 — ETL (Extract – Transform – Load)

1.3.1 Objectifs

- Construire un pipeline reproductible qui transforme les données brutes en données analytiques propres.
- Charger ces données dans une base analytique exploitable par la BI et le ML.

1.3.2 À faire

1. **Extract** : lecture des CSV et fichiers Excel.
2. **Transform** :
 - Conversion des dates (YYYYMMDD → `datetime`).
 - Traitement des valeurs NULL textuelles.
 - Jointures avec les tables de dimensions.
 - Calcul de variables dérivées (âge, ancienneté, etc.).
 - Déduplication et contrôle de cohérence.
3. **Load** : chargement dans le Data Warehouse cible.

1.3.3 Outils proposés

Trois familles d'outils sont acceptées pour l'intégration des données. Choisissez celle qui correspond le mieux aux compétences de l'équipe et aux contraintes de votre poste de travail.

Outil	Type	Forces	À considérer
Talend Open Studio	ETL graphique open source	Interface visuelle (drag & drop), nombreux connecteurs, génère du code Java, bonne traçabilité du flux	Installation lourde (JDK requis), démarrage plus long, version Open Studio non maintenue récemment
SSIS (SQL Server Integration Services)	ETL graphique Microsoft	Très bien intégré avec SQL Server et Power BI, mature en entreprise, performant sur grosse volumétrie	Nécessite Windows + Visual Studio + SQL Server Data Tools, écosystème Microsoft requis
Python (pandas / polars + SQLAlchemy / DuckDB)	ETL programmation	Flexible, reproductible, versionnable facilement, transition naturelle vers le ML	Pas d'interface graphique, demande de la rigueur dans la structuration du code

Comment choisir ?

- Vous visez l'écosystème Microsoft (Power BI + SQL Server) → **SSIS** est cohérent de bout en bout.
- Vous voulez une approche visuelle indépendante de la base cible → **Talend Open Studio**.
- Vous voulez la solution la plus rapide à mettre en place et compatible avec le ML qui suit → **Python**.

Outils complémentaires

Catégorie	Options
Transformation SQL avancée	dbt , DuckDB , SQL natif
Orchestrateur (optionnel, <i>overkill</i> sur 1 mois)	Apache Airflow, Prefect, Dagster
Alternative no-code	Pentaho Kettle, KNIME

Important : quel que soit l'outil retenu, votre pipeline doit être **reproductible** (un tiers doit pouvoir le ré-exécuter) et **documenté** par un schéma clair du flux de données.

1.4 Étape 3 — Data Warehouse et modélisation dimensionnelle (BI)

1.4.1 Objectifs

- Stocker les données préparées dans une base interrogeable.
- Modéliser en étoile : une table de faits + plusieurs tables de dimensions.

1.4.2 À faire

- Choisir la base de données cible.

- Modéliser la table de faits : grain = (client, compte, mois) ou (client, compte) selon votre choix.
- Définir les dimensions : `dim_client`, `dim_compte`, `dim_produit`, `dim_temps`, `dim_industrie`, `dim_devise`, `dim_motif_cloture`, etc.
- Créer les scripts SQL de création des tables.
- Charger les données depuis l'ETL.
- Rédiger un schéma visuel (diagramme en étoile).

1.4.3 Outils proposés

Catégorie	Options
Base de données analytique légère	DuckDB (excellent pour ce projet : zéro configuration), SQLite
Base de données SQL classique	PostgreSQL , MySQL , MariaDB
Base de données analytique avancée	ClickHouse , MS SQL Server (utile si vous visez Power BI sur site)
Modélisation dimensionnelle	dbt (transformation + tests), SQL natif
Visualisation du schéma	dbdiagram.io, draw.io, Lucidchart, Mermaid

Recommandation : **DuckDB** ou **PostgreSQL** sont les meilleurs choix pour ce projet. DuckDB pour la simplicité, PostgreSQL si vous voulez une expérience plus proche de la production.

1.5 Étape 4 — Tableaux de bord Power BI

1.5.1 Objectifs

- Restituer les KPIs métier de manière interactive.
- Permettre une exploration par profil utilisateur (direction, conseillers, marketing).

1.5.2 À faire

- Se connecter à la source de données (DWH).
- Définir les mesures DAX nécessaires (taux de churn, ancienneté moyenne, etc.).
- Construire au minimum **3 pages** :
 1. **Vue d'ensemble** : KPIs clés, taux de churn global, évolution.
 2. **Analyse du churn** : décomposition par segment (âge, secteur, ancienneté, produit, devise).
 3. **Segmentation client** : profils, valeur client, comptes à risque.
- Soigner la mise en page : filtres, slicers, navigation entre pages, infobulles.
- Publier le rapport (Power BI Service ou fichier `.pbix`).

1.5.3 Outils proposés

Catégorie	Options
Outil principal	Power BI Desktop (obligatoire — l'énoncé l'impose)

Catégorie	Options
Publication	Power BI Service (compte gratuit pour la publication, payant pour le partage)
Alternatives complémentaires	Tableau, Metabase, Apache Superset, Looker Studio (non — Power BI exigé)

Conseil : la qualité du design des dashboards compte. Inspirez-vous des galeries Power BI officielles. Ne surchargez pas vos pages.

1.6 Étape 5 — Machine Learning

1.6.1 Objectifs

- Construire un modèle qui prédit la probabilité de churn d'un client.
- Choisir le meilleur modèle de manière justifiée.

1.6.2 À faire

1. **Préparation** : sélectionner les features, encoder les variables catégorielles, gérer le déséquilibre (poids des classes, SMOTE, etc.), split train/test stratifié.
2. **Modélisation** : entraîner **au moins 3 modèles** différents (par exemple : régression logistique en baseline, Random Forest, XGBoost).
3. **Évaluation** : utiliser les métriques adaptées au déséquilibre (precision, recall, F1, **ROC-AUC**, **PR-AUC**). Pas seulement l'accuracy.
4. **Tuning** : optimiser les hyperparamètres du meilleur candidat.
5. **Interprétation** : importance des features + valeurs SHAP.
6. **Sauvegarde** : sérialiser le modèle final.

1.6.3 Outils proposés

Catégorie	Options
Manipulation	pandas , numpy
Modèles classiques	scikit-learn
Modèles boostés	XGBoost , LightGBM , CatBoost
Déséquilibre de classes	imbalanced-learn (SMOTE), scale_pos_weight
Hyperparamètres	GridSearchCV , RandomizedSearchCV , Optuna
Interprétabilité	SHAP , eli5 , importance native des arbres
Sauvegarde	joblib , pickle , formats ONNX ou MLflow
Suivi d'expériences (optionnel)	MLflow , Weights & Biases

1.7 Étape 6 — Interface web et déploiement

1.7.1 Objectifs

- Exposer le modèle ML et les analyses à des utilisateurs non-techniques.

- Déployer en ligne pour une démonstration accessible.

1.7.2 À faire

- Concevoir une interface simple avec au moins :
 - Un **formulaire de prédiction individuelle** (saisir un profil client → recevoir un score de churn).
 - Une **liste des clients actifs à risque** issue d'une prédiction en lot.
 - Des **KPIs de synthèse**.
- Brancher le modèle sauvegardé à l'application.
- Déployer sur une plateforme cloud accessible publiquement (URL fournie dans le README).

1.7.3 Outils proposés — Front + back unifié (recommandé pour ce projet)

Outil	Forces	Limites
Streamlit	Très rapide à développer en Python pur, idéal pour démo data	Personnalisation graphique limitée
Gradio	Très simple pour exposer un modèle ML	Moins polyvalent qu'un dashboard complet
Dash (Plotly)	Très bonnes visualisations, plus contrôlable	Courbe d'apprentissage un peu plus longue

1.7.4 Outils proposés — Front et back séparés (si vous voulez aller plus loin)

Couche	Options
API back-end	FastAPI (recommandé), Flask, Django REST
Front-end	React, Vue, Angular, Next.js, simple HTML/Bootstrap

1.7.5 Outils proposés — Hébergement / Déploiement

Option	Idéal pour
Streamlit Community Cloud	App Streamlit (gratuit, ultra simple)
Hugging Face Spaces	App Gradio / Streamlit (gratuit)
Render	App Python complète (free tier)
Railway	App Python + DB (free tier limité)
Vercel / Netlify	Front statique ou Next.js
Docker + un cloud (Azure, GCP, AWS)	Si vous maîtrisez Docker

Recommandation : Streamlit + Streamlit Cloud ou **Gradio + Hugging Face Spaces** pour un déploiement rapide et gratuit. Visez la simplicité.

1.8 Étape 7 — Rapport et présentation

1.8.1 Objectifs

- Documenter votre démarche, vos choix et vos résultats.
- Convaincre le jury en 20 minutes que votre solution répond au besoin.

1.8.2 Structure recommandée du rapport (25–40 pages)

1. **Page de garde et résumé exécutif** (1 page).
2. **Introduction et contexte** (2 pages).
3. **Description et exploration des données** (3–5 pages).
4. **Architecture de la solution** (2–3 pages, avec schémas).
5. **ETL et modélisation dimensionnelle** (3–5 pages).
6. **Analyses BI et dashboards Power BI** (3–5 pages, captures d’écran).
7. **Modélisation ML** (5–8 pages, métriques, comparaisons, interprétation).
8. **Interface web et déploiement** (2–3 pages, captures, lien démo).
9. **Limites et perspectives** (1–2 pages).
10. **Conclusion** (1 page).
11. **Annexes** : références, code clés, glossaire.

1.8.3 Outils proposés

Catégorie	Options
Rédaction du rapport	LaTeX (Overleaf), Microsoft Word , Google Docs , Typst , Markdown + Pandoc
Diagrammes	draw.io , Lucidchart , Mermaid , Excalidraw , dbdiagram.io
Captures d’écran	ShareX, Snipping Tool, macOS Screenshot
Slides	PowerPoint , Google Slides , Canva , Beamer (LaTeX), Marp (Markdown)

1.8.4 Conseils pour la présentation orale

- **Temps** : 20 minutes de présentation + 10 minutes de questions. Ne dépassez pas.
- **Rythme** : ~1 slide par minute.
- **Démo** : prévoyez **au moins 5 minutes** de démonstration live de votre application web. Préparez un plan B (vidéo de secours) en cas de problème de connexion.
- **Répartition** : chaque membre de l’équipe doit prendre la parole.
- **Public** : présentez à un public à la fois technique et métier — expliquez les choix techniques de manière accessible.

1.9 Récapitulatif — Stack recommandée (option simple et rapide)

Si vous voulez une stack qui “fonctionne”, voici une combinaison cohérente :

Composante	Choix recommandé
Langage	Python 3.10+

Composante	Choix recommandé
EDA / ETL	pandas + Jupyter
Data Warehouse	DuckDB ou PostgreSQL
Dashboards	Power BI Desktop
ML	scikit-learn + XGBoost
Interface web	Streamlit
Déploiement	Streamlit Community Cloud
Versioning	Git + GitHub
Rapport	Word ou LaTeX (Overleaf)
Slides	PowerPoint ou Google Slides

Cette stack permet de couvrir tout le périmètre en restant dans le temps imparti. Vous êtes libres d'en choisir une autre, **mais justifiez vos choix dans le rapport.**